Bioinformatics algorithms

Rosario M. Piro
Politecnico di Milano

Read mapping – exercises

---

1. **Suffix trie and suffix tree**

   Given the word "RAZZMATAZZ":[1]

   (a) Build a suffix trie for the word.

   (b) Compact the trie by combining non-branching paths.

   (c) Transform the compressed trie into a suffix tree.

   (d) *Use the graphs* to answer the following questions:

   1. How many times does the letter "A" occur?
   2. What is the longest repeated substring?
   3. At what positions in the original string does the longest repeated substring occur?

   (e) Remember the following algorithm to create a suffix trie (from the lectures):

Algorithm 1 SuffixTrie(T)

```
1: T+ = $
2: root = {}
3: for i=1 to i=length(T) do
4:    n = root{ # n is the current node}
5:    for c in T[i :] { # for each char in the i-th suffix} do
6:       if c ∉ n then
7:          n[c] = {} { # add outgoing edge to n if needed }
8:       end if
9:       n = n[c]{ # switch current node to child node }
10:   end for
11: end for
12: return root
```

   How can you change the algorithm to build a prefix trie?

   What if instead you want to build a suffix trie for the *reverse complement* of a nucleotide sequence?

[1] Meaning of "razzmatazz" from The Free Dictionary:
1. Excited action or impressive display.
2. Evasive or misleading language; double talk.

2. **Suffix array**
   (a) What is the suffix array (SA) for the word "RAZZMATAZZ"?
   (b) What is the selective SA for k=3 (every 3rd)?
   (c) Using the information from the suffix array, answer the following questions:
      1. At what positions does the letter "Z" occur in the original string?
      2. How can you use this information to identify the longest repeated substring?
   (d) (Optional; for fun; no solution here) Implement the Manber-Myers algorithm to sort a suffix array!

3. **Burrows-Wheeler transform**
   (a) Compute the Burrows-Wheeler transform (BWT) for the input string "`ACGTTGAACTT`" (without the quotation marks)
      - ... by performing rotations
      - ... by first computing the suffix array and from this the BWT
   (b) (Optional; for fun; no solution here) Write a program that takes an input string and returns it's BWT. No need for efficiency. Test your program with the input string from above! If you want a more challenging task: write a program which first computes the suffix array using the Manber-Myers sorting and then converts the SA into the BWT.

4. **Burrows-Wheeler transform 2**
   (a) Given the BWT="`CGGTCGAATGA$ATTTTA`" (without the quotation marks), what is the original string? First use a "graphical approach" ...
   (b) For the BWT above (BWT = `CGGTCGAATGA$ATTTTA`), define the following additional data structures:
      1. For the F-column: the vector $C(a)$ of cumulative counts for all symbols $a \in \Sigma$ (here: all nucleotides)
      2. An individual vector $Occ(i) = Occ(BWT[i], i)$ counting the number of occurrences of character $a = BWT[i]$ until position i in the BWT, where $BWT[i]$ is precisely the character at that position (e.g., if $BWT[i]$ is "T", then $Occ(i) = Occ(BWT[i], i)$ is the number of "T"s in BWT[0:i])
         Note: this is similar to the tally table, but the information is summarized in one vector!
   (c) Now write a short algorithm (pseudo code) which uses these two additional data structures, along with the BWT, to reconstruct the original string T!
   (d) In the lecture we have seen, that for searching occurrences of substrings (e.g., reads in the genome), we would use the FM index, composed of the BWT, a representation of the F-column, a (selective) suffix array and the tally table (checkpoints). Using instead only the structures we have so far defined in this exercise (BWT, $C(a)$ and $Occ(BWT[i], i)$), would it be possible to
      1. ... determine the number of occurrences of a substring?
      2. ... identify their positions within the original string T?

      If so, how? (A conecptual description is enough, no precise algorithm needs to be developed)

5. **FM index**
   (a) For the following input string, build the full FM index with k=3 for the selective SA and x=4 for the tally table checkpoints:

      "razzledazzle" (without quotation marks)

   (b) Use the data structures from the FM index to search for the occurrences and positions of the substring "azzl" ... Here, report the evolving index ranges for the single steps that you obtain for the partial solutions and the final positions within the original string

6. **Inexact matching**

   Given the original string T="razzledazzle$" and the alphabet Σ=(a,d,e,l,r,z):

(a) Precalculate the lower bounds $D(i)$ of mismatches for the search word W="hazle".

(b) Consider the algorithm `InexRecur(W,i,z,k,l)` used to recursively find possible matches allowing for up to d_{max} errors. Explain how you can use $D(i)$, computed above, to early interrupt unsuccessful mapping attempts for searching W="hazle" with $d_{max} = 1$.

(c) Given $D(i)$, computed above, execute a full search of W="hazle" with $d_{max} = 2$. No need to compute all SA intervals (lines 9-10 of the algorithm), just explain what happens in the individual iterations/steps!